5 · Physics-Informed Neural Networks (PINNs)

Instead of discretizing a PDE on a mesh, a **Physics-Informed Neural Network** trains a neural network $u_\theta(x)$ to satisfy the PDE's residual (and its boundary conditions) at randomly sampled collocation points. `NeuralPDE.jl` turns a symbolic PDE specification (written with `ModelingToolkit.jl`) directly into an `Optimization.jl` training problem.

We solve the 1D Poisson equation

$$u''(x) = f(x) = -\pi^2 \sin(\pi x), \qquad u(0) = 0,\ u(1) = 0$$

whose exact solution is $u(x) = \sin(\pi x)$, so we can check the network's accuracy directly.

Error message

Multiple expressions in one cell

How would you like to fix it?

- Split this cell into 2 cells
- Wrap all code in a *begin ... end* block.

```
using NeuralPDE, Lux, ModelingToolkit, Optimization, OptimizationOptimisers, Random, Plots
import ModelingToolkit: Interval
```

1. Symbolic problem specification

`ModelingToolkit.jl` lets us write down the PDE almost exactly as it looks on paper.

Error message

> **UndefVarError:** `ModelingToolkit` **not defined in this notebook.**
> Suggestion: check for spelling errors or missing imports.

Show stack trace...

```
begin
    ModelingToolkit.@parameters x
    ModelingToolkit.@variables u(..)
    Dxx = ModelingToolkit.Differential(x)^2

    f(x) = -π^2 * sin(π * x)
    eq = Dxx(u(x)) ~ f(x)

    bcs = [u(0.0) ~ 0.0, u(1.0) ~ 0.0]
    domains = [x ∈ Interval(0.0, 1.0)]
end
```

2. The neural network ansatz

A small `Lux` network stands in for $u_\theta(x)$. `NeuralPDE.jl` handles differentiating it (via automatic differentiation) to build the residual loss automatically.

Error message

> **UndefVarError:** `Random` **not defined in this notebook.**
> Suggestion: check for spelling errors or missing imports.
> Hint: Random is loaded but not imported in the active module Main.

Show stack trace...

```
begin
    rng5 = Random.default_rng()
    Random.seed!(rng5, 0)
    chain = Chain(Dense(1, 16, tanh), Dense(16, 16, tanh), Dense(16, 1))
end
```

Error message

> **UndefVarError:** `PhysicsInformedNN` not defined in this notebook.
> Suggestion: check for spelling errors or missing imports.

```
discretization = PhysicsInformedNN(chain, QuadratureTraining())
```

Error message

> **UndefVarError:** `@named` not defined in this notebook.
> Suggestion: check for spelling errors or missing imports.

```
begin
    @named pde_system = PDESystem(eq, bcs, domains, [x], [u(x)])
    prob_pinn = discretize(pde_system, discretization)
end
```

3. Train the network to minimize the PDE + boundary-condition residual

Error message

> **UndefVarError:** `OptimizationOptimisers` not defined in this notebook.
> Suggestion: check for spelling errors or missing imports.

```
begin
    pinn_losses = Float64[]
    pinn_callback = function (state, loss)
        push!(pinn_losses, loss)
        return false
    end

    res_pinn = Optimization.solve(prob_pinn, OptimizationOptimisers.Adam(0.01);
    callback=pinn_callback, maxiters=1500)
end
```

Error message

> **UndefVarError: `plot` not defined in this notebook.**
> Suggestion: check for spelling errors or missing imports.

```
plot(pinn_losses; xlabel="iteration", ylabel="PDE + BC residual loss", yscale=:log10,
label="training loss")
```

4. Compare against the exact solution

Error message

> *Another cell defining `chain` or `prob_pinn` contains errors.*

```
begin
    phi = discretization.phi
    final_ps = res_pinn.u

    xs = 0.0:0.01:1.0
    u_predicted = [first(phi([xi], final_ps)) for xi in xs]
    u_exact = sin.(π .* xs)

    plot(xs, u_exact; label="exact  u(x) = sin(πx)", lw=2, xlabel="x", ylabel="u")
    plot!(xs, u_predicted; label="PINN", lw=2, ls=:dash)
end
```

Error message

> *Another cell defining `chain`, `prob_pinn`, or `u_predicted` contains errors.*

```
maximum(abs.(u_predicted .- u_exact))
```

Takeaways

- A PINN needs no mesh: the loss is just "how well does the network satisfy the equation and boundary conditions," evaluated at sampled points.

- `ModelingToolkit.jl` symbolic expressions let `NeuralPDE.jl` build the residual and its derivatives automatically via automatic differentiation, rather than requiring you to hand-derive them.
- The same recipe scales to higher-dimensional PDEs and systems of PDEs where classical mesh-based methods become expensive — the main change is a bigger network and more training time, not new mathematics in the notebook.